
Título : Revisão p1 BDII

Data : 2026-05-06

Tags : [#bd2](#)

Fonte: [PL-SQL.odt](#)

Conceitos

Blocos PL/SQL (DECLARE / BEGIN / EXCEPTION / END)

O que é: A unidade básica de código PL/SQL.

- **DECLARE** (opcional): variáveis, cursores, exceções definidas pelo usuário
- **BEGIN** (obrigatório): instruções SQL e instruções PL/SQL
- **EXCEPTION** (opcional): ações quando ocorrem erros
- **END;** (obrigatório)

Exemplo:

```
DECLARE
    v_variable varchar2(5);
BEGIN
    SELECT column_name
    INTO    v_variable
    FROM    table_name;

    EXCEPTION
        WHEN exception_name THEN
            NULL;
END;
/
```

Declaração de variáveis

O que é: Todos os identificadores PL/SQL devem ser declarados na seção **DECLARE** antes de serem referenciados no bloco.

Sintaxe:

```
identificador [CONSTANT] tipo_de_dados [NOT NULL] [:= | DEFAULT expr];
```

Exemplos:

```
DECLARE
  v_hiredate  DATE;
  v_deptno    NUMBER(2) NOT NULL := 10;
  v_location  VARCHAR2(13) := 'Erechim';
  c_comm      CONSTANT NUMBER := 1400;
  v_mgr       NUMBER(4) DEFAULT 7839;
BEGIN
  NULL;
END;
/
```

Tipos de dados básicos

Tipo	Descrição
VARCHAR2	Caracteres variável (até 32.767 bytes)
NUMBER	Números fixos e de ponto flutuante
DATE	Datas e horas (4712 AC a 9999 DC)
CHAR	Caracteres tamanho fixo (até 32.767 bytes)
BOOLEAN	TRUE , FALSE ou NULL
PLS_INTEGER	Inteiros (-2.147.483.647 a 2.147.483.647) — mais rápido que NUMBER
BINARY_INTEGER	Inteiros com mesma faixa do PLS_INTEGER
LONG / LONG RAW	Texto/dados binários grandes (legado)

Exemplo:

```
DECLARE
  v_contador PLS_INTEGER := 0;
  v_preco    NUMBER(7,2) := 123.45;
  v_nome     VARCHAR2(30) := 'MILLER';
BEGIN
  v_contador := v_contador + 1;
```

```
END;  
/
```

Atributo %TYPE

O que é: Declara uma variável com o mesmo tipo de uma coluna do banco ou de outra variável já declarada.

Quando usar: Quando a variável representa diretamente uma coluna, evitando divergência de tipo entre código e schema.

Exemplo:

```
DECLARE  
  v_ename      emp.ename%TYPE;  
  v_balance     NUMBER(7,2);  
  v_min_balance v_balance%TYPE;  
BEGIN  
  NULL;  
END;  
/
```

LOBs (Large Objects)

O que é: Variáveis para armazenar blocos de dados não estruturados (texto grande, imagens, vídeos etc.).

Tipo	Descrição
CLOB	Character Large Object — grandes textos
BLOB	Binary Large Object — dados binários (imagens, vídeos, áudios)
BFILE	Ponteiro para arquivo binário fora do banco (no filesystem)
NCLOB	Como CLOB, mas para conjuntos de caracteres internacionais (NCHAR)

Metáfora: BLOB/CLOB/NCLOB guardam o objeto dentro do cofre (BD); BFILE guarda só o "endereço do cofre externo".

Exemplo:

```
DECLARE
    v_texto CLOB;
BEGIN
    v_texto := 'Texto muito grande...';
END;
/
```

SELECT ... INTO (consulta de linha única)

O que é: Sintaxe do `SELECT` em PL/SQL com `INTO` para jogar colunas em variáveis. Funciona **somente quando retorna uma linha** — zero ou mais de uma linha geram erro (`NO_DATA_FOUND` / `TOO_MANY_ROWS`).

Sintaxe:

```
SELECT col1, col2
INTO    variavel1, variavel2
FROM    tabela
WHERE   condicao;
```

DML dentro de PL/SQL (INSERT / UPDATE / DELETE)

O que é: Executar `INSERT`, `UPDATE`, `DELETE` dentro do bloco PL/SQL como parte de uma lógica maior.

Inserindo:

```
BEGIN
    insert into emp (empno, ename, job, deptno)
    values (999, 'HARDING', 'CLERK', 10);
END;
/
commit;
```

Atualizando:

```
DECLARE
    v_sal_increase emp.sal%type := 2000;
BEGIN
```

```

update emp
set     sal = sal + v_sal_increase
where  job = 'ANALYST';
END;
/

```

Excluindo:

```

DECLARE
  v_ename emp.ename%type := 'JAMES';
BEGIN
  delete from emp
  where  ename = v_ename;
END;
/

```

COMMIT / ROLLBACK / SAVEPOINT

O que é: Controle de transações DML. Transações iniciam e terminam com `COMMIT` ou `ROLLBACK`. `SAVEPOINT` marca um ponto intermediário.

Exemplo com COMMIT no bloco:

```

DECLARE
  v_sal_increase emp.sal%type := 500;
BEGIN
  update emp
  set     sal = sal + v_sal_increase
  where  job = 'ANALYST';

  commit;
END;
/

```

Exemplo com SAVEPOINT e ROLLBACK:

```

BEGIN
  savepoint antes_do_update;

  update emp
  set     sal = sal * 1.10
  where  job = 'CLERK';

```

```
-- desfaz até o savepoint:
rollback to antes_do_update;

-- ou desfaz tudo:
-- rollback;
END;
/
```

Procedures

O que é: Conjunto de instruções SQL armazenadas no BD e executadas quando invocadas. Pode aceitar parâmetros de entrada e/ou retorno de valores. Encapsula lógica repetida.

Quando usar: Quando a aplicação precisa chamar algo "pronto" no BD sem reescrever script.

Functions

O que é: Similar à procedure, mas **retorna um valor escalar**. Pode ser chamada em SQL ou dentro de blocos PL/SQL.

Uso em SQL:

```
select f_menor_sal('ANALYST') from dual;

select ename, job, sal
from emp
where sal = f_menor_sal('CLERK');
```

Uso dentro de bloco PL/SQL:

```
DECLARE
    v_menor_sal number(7,2);
BEGIN
    v_menor_sal := f_menor_sal('CLERK');

    UPDATE emp
    SET     sal = v_menor_sal
    WHERE   job = 'CLERK';

    COMMIT;
```

```
END;  
/
```

Código compilado fica na view `user_source` :

```
SELECT text FROM user_source  
WHERE name = 'F_MENOR_SAL' and type = 'FUNCTION';
```

Triggers

O que é: Conjunto de instruções automaticamente executadas em resposta a eventos no BD (INSERT , UPDATE , DELETE), antes (BEFORE) ou depois (AFTER). Usada para auditoria, integridade, validação etc.

Metáfora: Sensor de presença: evento acontece → reação automática.

Quando usar: Quando a ação **tem que acontecer sempre** que o evento ocorrer, independente de quem fez (app, script, usuário).

Cursoros implícitos

O que é: Toda instrução SQL abre uma área de memória (cursor). Em PL/SQL, o cursor implícito é criado e gerenciado automaticamente para cada SQL executado.

Metáfora: Uma "janela" temporária onde o Oracle coloca o resultado/efeito do último comando.

Atributos:

Atributo	Descrição
SQL%ROWCOUNT	Número de linhas afetadas pela instrução SQL mais recente
SQL%FOUND	TRUE se afetou 1+ linhas
SQL%NOTFOUND	TRUE se não afetou linhas
SQL%ISOPEN	Sempre FALSE para implícitos (PL/SQL fecha automaticamente)

Estruturas de controle – IF / ELSIF / ELSE

O que é: Altera o fluxo lógico com condicionais.

Metáfora: Semáforo de decisões: dependendo da condição, você segue por uma rua diferente.

Sintaxe:

```
IF condicao THEN
    instrucoes;
ELSIF outra_condicao THEN
    instrucoes;
ELSE
    instrucoes;
END IF;
```

Exemplos:

```
IF v_ename = 'OSBORNE' THEN
    v_mgr := 22;
END IF;
```

```
IF v_ename = 'MILLER' THEN
    v_job      := 'SALESMAN';
    v_deptno   := 35;
    v_new_comm := sal * 0.20;
END IF;
```

Loops (Básico / FOR / WHILE)

O que é: Repetição de instruções até uma condição ser satisfeita.

Metáfora: Linha de montagem: repete o mesmo passo até uma condição parar.

Loop básico:

```
LOOP
    -- instrucoes
    EXIT WHEN condicao;
END LOOP;
```

Loop FOR:


```
FOR contador IN [REVERSE] lower_bound .. upper_bound LOOP
    -- instrucoes
END LOOP;
```

Loop WHILE:

```
WHILE condicao LOOP
    -- instrucoes
END LOOP;
```

Atributo %ROWTYPE

O que é: Declara um registro com base em todas as colunas de uma tabela ou view.

Metáfora: Pegar uma "ficha completa" do funcionário (linha inteira), em vez de carregar campo por campo.

Quando usar: Quando quer trabalhar com várias colunas e evitar declarar N variáveis.

Cursorres explícitos (OPEN / FETCH / CLOSE)

O que é: Cursor declarado e nomeado pelo programador, dando controle total de iteração e processamento linha a linha.

Declaração:

```
CURSOR cursor_name IS
    select_statement;
```

Operações:

```
OPEN cursor_name;
FETCH cursor_name INTO variavel1, variavel2;
CLOSE cursor_name;
```

Atributos:

Atributo	Descrição
%ISOPEN	TRUE se o cursor está aberto
%FOUND	TRUE se o último FETCH retornou linha
%NOTFOUND	TRUE se o último FETCH não retornou linha
%ROWCOUNT	Número de linhas extraídas até o momento

Cursor com registros (cursor%ROWTYPE)

O que é: Usar record baseado no cursor para não precisar listar todas as variáveis no FETCH .

FOR LOOP de cursor (atalho automático)

O que é: Atalho onde o cursor é aberto, extraído linha a linha e fechado automaticamente. Percorre **todo** o cursor.

Sintaxe:

```
FOR record_name IN cursor_name LOOP
    -- instrucoes
END LOOP;
```

Cursor com parâmetros

O que é: Parâmetros passados na abertura do cursor e usados na consulta.

Sintaxe:

```
CURSOR cursor_name (parameter_name tipo_de_dado, ...) IS
    select_statement;
```

Exemplo:

```
DECLARE
    CURSOR emp_cursor (p_deptno NUMBER, p_job VARCHAR2) IS
        SELECT empno, ename, job FROM emp
```

```
WHERE deptno = p_deptno AND job = p_job;
BEGIN
  FOR emp_record IN emp_cursor(10, 'ANALYST') LOOP
    DBMS_OUTPUT.PUT_LINE(emp_record.ename);
  END LOOP;
END;
/
```

Cláusula FOR UPDATE

O que é: Garante que as linhas estejam liberadas para atualização antes de disparar o UPDATE . NOWAIT evita espera indefinida.

Exemplo:

```
DECLARE
  CURSOR emp_cursor IS
    select empno, ename, sal
    from emp
    where deptno = 30
    FOR UPDATE OF sal NOWAIT;
BEGIN
  NULL;
END;
/
```

Cursores com subconsultas (na FROM)

O que é: Cursor que usa subconsulta para montar um conjunto temporário e filtrar/combinar.

Metáfora: Fazer uma "pré-lista" (tabela temporária lógica) e depois usar ela como insumo.

Exemplo:

```
DECLARE
  CURSOR funcs_cursor IS
    select t1.deptno, t1.dname, t2.total
    from dept t1,
         (select deptno, count(*) total
          from emp
          group by deptno) t2
```

```
        where t1.deptno = t2.deptno
           and t2.total > 3;
BEGIN
    NULL;
END;
/
```

Tratamento de exceções

O que é: Exceção é um identificador criado durante execução que termina o corpo principal do bloco. Pode ser gerada pelo Oracle ou explicitamente com `RAISE`.

Metáfora: Airbag do código: algo inesperado acontece e você tem um mecanismo de proteção para não "capotar" todo o sistema.

Sintaxe:

```
EXCEPTION
  WHEN excecao1 [OR excecao2 ...] THEN
    instrucoes;
  WHEN OTHERS THEN
    instrucoes;
```

| `WHEN OTHERS` captura qualquer exceção não tratada — sempre por último.

Exceções pré-definidas:

Nome	Erro Oracle	Quando ocorre
<code>NO_DATA_FOUND</code>	ORA-01403	SELECT INTO sem retorno de dados
<code>TOO_MANY_ROWS</code>	ORA-01422	SELECT INTO retorna mais de uma linha
<code>ZERO_DIVIDE</code>	ORA-01476	Divisão por zero
<code>CURSOR_ALREADY_OPEN</code>	ORA-06511	Tentativa de abrir cursor já aberto
<code>INVALID_NUMBER</code>	ORA-01722	Falha na conversão de string para número
<code>LOGIN_DENIED</code>	ORA-01017	Login com usuário/senha inválido
<code>TOO_MANY_ROWS</code>	ORA-01422	SELECT INTO retorna mais de uma linha

Exercícios

Exercício 1

Construir um bloco PL-SQL que retorna o número do departamento e sua localização.

```
DECLARE
    v_deptno number(2);
    v_loc     varchar2(15);
BEGIN
    select deptno, loc
    into    v_deptno, v_loc
    from    dept
    where   dname = 'SALES';
END;
/
```

Exercício 2

Mesmo do exercício 1, mas exibindo a informação com `dbms_output`.

```
set serverout on
DECLARE
    v_deptno number(2);
    v_loc     varchar2(15);
BEGIN
    select deptno, loc
    into    v_deptno, v_loc
    from    dept
    where   dname = 'SALES';

    dbms_output.put_line('Departamento: ' || to_char(v_deptno) || '-' ||
'Localizacao: ' || v_loc);
END;
/
```

Exercício 3

Recuperar o total de salários de todos os funcionários no departamento especificado.

```
DECLARE
    v_sum_sal emp.sal%type;
    v_deptno  number not null := 10;
BEGIN
```

```

select sum(sal)
into v_sum_sal
from emp
where deptno = v_deptno;

dbms_output.put_line('Soma dos salarios: ' || to_char(v_sum_sal));
END;
/

```

Procedures

Exercício 4

Criar uma procedure que receba um número de departamento e exiba a localização.

```

CREATE OR REPLACE PROCEDURE p_local (v_deptno number) as
  v_loc varchar2(15);
BEGIN
  select loc
  into v_loc
  from dept
  where deptno = v_deptno;

  dbms_output.put_line('Depto num: ' || to_char(v_deptno) || ' - ' ||
'Local: ' || v_loc);
END;
/

```

```

-- Executar:
set serverout on
exec p_local(10)

-- Ver erros de compilação:
show errors

```

Functions

Exercício 5

Criar uma função que receba o cargo (job) e retorne o menor salário do cargo informado.

```

CREATE OR REPLACE FUNCTION f_menor_sal (v_job varchar2) RETURN NUMBER as
    v_menor_sal number(7,2);
BEGIN
    select min(sal)
    into    v_menor_sal
    from    emp
    where   job = v_job;

    RETURN v_menor_sal;
END;
/

```

Triggers`

Exercício 6 — Trigger de auditoria salarial

Registrar alterações de salário na tabela ATUALIZACOES_SALARIOS .

```

create table ATUALIZACOES_SALARIOS (
    empno          number(4),
    ename          varchar2(10),
    sal_antes      number(7,2),
    sal_depois     number(7,2),
    data_alteracao date
);
/

CREATE OR REPLACE TRIGGER t_registra_salario
BEFORE UPDATE OF sal ON emp
FOR EACH ROW
BEGIN
    IF :OLD.sal != :NEW.sal THEN
        INSERT INTO ATUALIZACOES_SALARIOS
            (empno, ename, sal_antes, sal_depois, data_alteracao)
        VALUES
            (:OLD.empno, :OLD.ename, :OLD.sal, :NEW.sal, SYSDATE);
    END IF;
END;
/

```

Teste:

```
update emp set sal=10000 where job='CLERK';
commit;

select * from atualizacoes_salarios;
```

⚠ Sem COMMIT, nem o update nem os registros na tabela auxiliar efetivam.

Exercício 7 e 8 — Dependências e status INVALID

Dropar a tabela ATUALIZACOES_SALARIOS deixa a trigger como INVALID.

```
select * from user_dependencies where name = 'T_REGISTRA_SALARIO';
select * from user_objects where status = 'INVALID';

-- Recompilar:
alter trigger T_REGISTRA_SALARIO compile;
show errors
```

Cursor implícitos

Exercício 9

Excluir do depto 10 e mostrar quantas linhas foram excluídas; usar rollback (é só teste).

```
set serverout on size 10000
DECLARE
    v_deptno number := 10;
BEGIN
    delete from emp
    where deptno = v_deptno;

    dbms_output.put_line('Foram excluidas ' || to_char(SQL%ROWCOUNT) || '
linhas da tabela EMP');
    rollback;
END;
/
```

Atributo %ROWTYPE

Exercício 16

Copiar dados do funcionário MILLER para tabela `retired_emp`.

```
create table retired_emp as
select * from emp
where rownum < 1;
/

DECLARE
    emp_rec emp%rowtype;
BEGIN
    select *
    into    emp_rec
    from    emp
    where   empno = &numero_do_empregado;

    insert into retired_emp
        (empno, ename, job, mgr, hiredate, leavedate, sal, comm, deptno)
    values
        (emp_rec.empno, emp_rec.ename, emp_rec.job, emp_rec.mgr,
emp_rec.hiredate,
        sysdate, emp_rec.sal, emp_rec.comm, emp_rec.deptno);

    commit;
END;
/
```

[`Cursors explícitos \(OPEN / FETCH / CLOSE\)`](#)

Exercício 17

Recuperar os 10 primeiros funcionários da tabela EMP.

```
set serverout on
DECLARE
    v_empno emp.empno%TYPE;
    v_ename emp.ename%TYPE;

    CURSOR emp_cursor IS
        select empno, ename
        from    emp;
BEGIN
    OPEN emp_cursor;
```

```

FOR i IN 1..10 LOOP
    FETCH emp_cursor INTO v_empno, v_ename;
    DBMS_OUTPUT.PUT_LINE('Funcionario ' || to_char(i) || ':' || v_empno ||
'- ' || v_ename);
END LOOP;

CLOSE emp_cursor;
END;
/

```

Exemplo com %NOTFOUND e %ROWCOUNT:

```

set serverout on
DECLARE
    v_empno emp.empno%type;
    v_ename emp.ename%type;

    CURSOR emp_cursor IS
        SELECT empno, ename FROM emp;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_empno, v_ename;
        EXIT WHEN emp_cursor%rowcount > 6 OR emp_cursor%notfound;
        DBMS_OUTPUT.PUT_LINE('Funcionario: ' || v_empno || '- ' || v_ename);
    END LOOP;

    CLOSE emp_cursor;
END;
/

```

`Cursors com registros (cursor%ROWTYPE)`

Exercício 18

Recuperar empno, ename e inserir em tabela ListaFunc.

```

create table ListaFunc (
    NumFunc number(4),
    NomeFunc varchar2(10)
);
/

```

```

DECLARE
  CURSOR emp_cursor IS
    SELECT empno, ename FROM emp;

  emp_record emp_cursor%ROWTYPE;
BEGIN
  OPEN emp_cursor;

  LOOP
    FETCH emp_cursor INTO emp_record;
    EXIT WHEN emp_cursor%NOTFOUND;

    INSERT INTO ListaFunc (numfunc, nomefunc)
    VALUES (emp_record.empno, emp_record.ename);
  END LOOP;

  COMMIT;
  CLOSE emp_cursor;
END;
/

```

[`FOR LOOP de cursor \(atalho automático\)`](#)

Exercício 19

Mostrar funcionários que trabalham no depto de vendas (deptno=30).

```

set serverout on
DECLARE
  CURSOR emp_cursor IS
    select ename, deptno from emp;
BEGIN
  FOR emp_record IN emp_cursor LOOP
    IF emp_record.deptno = 30 THEN
      DBMS_OUTPUT.PUT_LINE('0 funcionario ' || emp_record.ename || '
trabalha no depto de vendas');
    END IF;
  END LOOP;
END;
/

```

Com subconsulta (sem declarar cursor):

```

set serverout on
BEGIN
  FOR emp_record IN (select ename, deptno from emp) LOOP
    IF emp_record.deptno = 30 THEN
      DBMS_OUTPUT.PUT_LINE('O funcionario ' || emp_record.ename || '
trabalha no depto de vendas');
    END IF;
  END LOOP;
END;
/

```

Tratamento de exceções

Exercício 21

Testar com `PRESIDENT` (ok), `SALESMAN` (too many rows), `DBA` (no data found).

```

set serverout on
DECLARE
  v_ename emp.ename%type;
  v_deptno emp.deptno%type;
BEGIN
  SELECT ename, deptno
  INTO   v_ename, v_deptno
  FROM   emp
  WHERE  job = '&1';

  DBMS_OUTPUT.PUT_LINE('Funcionario ' || v_ename || ' trabalha no depto ' ||
v_deptno);

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Erro: nenhum dado encontrado pela consulta SQL');
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Mais de uma linha encontrada pela consulta SQL');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erro: ' || SQLERRM);
END;
/

```

Exercícios PL/SQL prova

Exercício 6

Crie um bloco PL/SQL ou uma stored procedure que recebe um número de matrícula (empno) e imprime 'diretor presidente' se o cargo (job) for 'PRESIDENT', ou 'funcionário' caso contrário.

```
CREATE OR REPLACE PROCEDURE p_classifica_emp (p_empno emp.empno%TYPE) AS
    v_job emp.job%TYPE;
BEGIN
    SELECT job
    INTO    v_job
    FROM    emp
    WHERE   empno = p_empno;

    IF v_job = 'PRESIDENT' THEN
        DBMS_OUTPUT.PUT_LINE('diretor presidente');
    ELSE
        DBMS_OUTPUT.PUT_LINE('funcionario');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Funcionario nao encontrado.');
```

END;

/

-- Executar:

```
set serverout on
exec p_classifica_emp(7839)
```

Exercício 7

Escreva um bloco PL/SQL ou uma stored procedure que utilize um cursores para percorrer todos os funcionários do departamento 10 e exiba seus nomes e salários

```
declare
    cursor c_func is
    select ename, sal
    from emp
    where deptno = 10;
```

```

        c_record c_func%rowtype;
begin
    open c_func;
    loop
        fetch c_func into c_record; -- primeira linha que o cursor c_func
retorna
        exit when c_func%notfound -- quando acabar os dados

        dbms_output.put_line('Nome: ' || c_record.ename || ' Salario:
' || c_record.sal ||)
    end loop;
    close c_func

end;

```

Exercicio 8

Crie um bloco PL/SQL ou uma stored procedure que tente buscar o salário de um funcionário pelo número (empno) passado por parâmetro. Se o funcionário não existir, capture a exceção e exiba a mensagem: 'Funcionário não encontrado.'. Caso contrário mostre o salário dele.

```

declare
    v_sal emp.sal%type;
begin
    select sal
    into v_sal
    from emp
    where empno = &emp_number; -- para passar parametro fora de procedure,
usa &

    dbms_output.put_line('Salário: ' || v_sal ||);

    exception
        when no_data_found then
            dbms_output.put_line('Funcionário não encontrado');
end;
/

```

Exercicio 9

Crie uma stored procedure chamada aumenta_salario que receba dois parametros: o número do funcionário(p_empno) e o percentual do aumento(p_percentual). A procedure

deve atualizar o salário do funcionário com base no percentual informado.

```
create or replace procedure aumenta_salario(p_empno emp.empno%type,  
p_percentual emp.sal%type) as  
    v_novo_sal emp.sal%type;  
begin  
    update emp  
    set sal = sal + sal * p_percentual/100  
    where empno = p_empno;  
end;
```